

An Ablation study on Normalized Vision Transformers

Pratyush Kumar
Stony Brook University

pratkumar@cs.stonybrook.edu

Sachin Kumar Danisetty
Stony Brook University

sdanisetty@cs.stonybrook.edu

Abstract

This study presents a comprehensive ablation analysis comparing the performance of a novel Vision Transformer (ViT) architecture based on normalized transformers with standard ViT models. Our research explores the efficacy of this new approach across a diverse range of computer vision tasks and datasets, including image classification on MNIST, CIFAR-10, CIFAR-100, and Tiny ImageNet. The normalized transformer, a recent innovation in transformer architecture, has shown promise in various natural language processing tasks. Our work extends its application to the domain of computer vision, hypothesizing that its unique properties may lead to improved performance and efficiency on vision tasks. We conduct a series of carefully designed ablation studies to isolate the impact of the normalized components on the ViT's performance. Our experiments evaluate model accuracy, computational efficiency, and scalability across datasets of varying complexity.¹

Preliminary results indicate that the normalized transformer-based ViT [2] demonstrates competitive performance compared to standard ViT architectures especially in terms of convergence speed and stability of our training process. Our findings have implications for the design of future vision models, particularly in scenarios where efficient processing of complex visual information is crucial.

1. Introduction

Normalization is one of the fundamental components in the training of neural network architectures. It reduces the effect of differences in the scale of input features, which can lead to some features dominating the learning process. Such dominance can cause models to focus on a subset of features, resulting in poor performance. By balancing inputs and reducing the impact of outliers, normalization improves convergence during optimization by keeping weights and

activations within a reasonable range. This mitigates issues like vanishing or exploding gradients, which can slow down or even prevent learning. Furthermore, normalization enhances optimization stability by reducing sensitivity to changes in inputs or weights, thereby improving generalization and robustness against overfitting.

Batch Normalization (BN) [3] is one of the most widely adopted normalization techniques in neural networks. BN normalizes activations by computing channel-wise mean and variance across the batch dimension, ensuring that activations have zero mean and unit variance. This stabilization accelerates convergence and makes optimization more robust to suboptimal hyperparameters such as learning rates or initialization schemes. BN has been particularly effective in computer vision tasks, where it contributes to stable training.

Despite its success, BN exhibits several limitations. It struggles with small batch sizes, as seen in tasks like image segmentation where memory constraints often necessitate small batches. In such cases, inaccurate estimates of mean and variance can degrade performance [9]. Additionally, BN introduces computational overhead during training since it requires mean-variance computation for each batch. Its dependency on batch size also complicates its application in scenarios with variable batch sizes.

To address these challenges, alternative normalization techniques have been proposed. Layer Normalization (LN) [1] normalizes activations for each individual sample by computing statistics across features rather than samples. LN is batch-size agnostic, making it suitable for applications with varying or small batch sizes, such as NLP tasks using recurrent neural networks (RNNs). LN simplifies inference by eliminating the need to track training statistics and reduces gradient instability by keeping activations within a reasonable range.

However, LN has its own drawbacks. It may underperform compared to BN for large batch sizes due to less stable statistics when normalizing across features instead of samples. Additionally, LN may struggle with architectures containing a large number of parameters or very deep layers due to its inability to capture complex activation patterns

¹All the code for this project is available at https://github.com/Cloaked04/cse527_finalProject.

effectively.

There is growing evidence that representation learning on the hypersphere is associated with more stable training, greater separability of the embedding space, and better performance on downstream tasks. NVIDIA has recently developed a new method for normalizing transformers with representation learning on the hypersphere called normalised transformers (nGPT) [7].

2. Normalized Transformer (nGPT)

The Normalized Transformer architecture introduces several key modifications to the standard Transformer model to enhance token similarity estimation and representation learning on the hypersphere. These modifications address fundamental issues in the original architecture and offer potential improvements in training stability and performance.

In the standard Transformer, the initial step involves calculating token similarities to retrieve corresponding embeddings. However, the unconstrained nature of both input and output embedding matrices leads to potential inaccuracies in similarity estimations. To mitigate this issue, nGPT implements a normalization procedure for embedding vectors after each iteration step. This normalization ensures that all dot products between embeddings are bounded within the range $[-1, 1]$, effectively representing cosine similarities[1].

A significant innovation in nGPT is the reframing of embedding transformations. Rather than simply adding embeddings to the outputs of self-attention and MLP blocks, the authors introduce spherical linear interpolations between embeddings and block outputs. This approach incorporates a learnable interpolation parameter α , termed the "eigen learning rate," which allows for more nuanced control over the transformation process.

The attention mechanism in nGPT undergoes substantial modifications. All matrices involved in the attention computation (query, key, and value) are normalized along their embedding dimensions. This normalization ensures that dot products between these unit norm vectors are bounded within $[-1, 1]$, representing cosine similarities. Consequently, attention matrices can be conceptualized as collections of normalized embedding vectors for comparison. To further control the norm of the total vectors, the authors propose an additional normalization step for query and key matrices. To compensate for the changes introduced by normalization and restore a variance of 1, the softmax scaling factor is adjusted from $1/\sqrt{d_k}$ to $\sqrt{d_k}$.

The final significant modification in nGPT involves the normalization of MLP weight matrices along the embedding dimension. This transformation ensures that the resulting vectors represent cosine similarities between the input vectors and those stored in the weight matrices. This approach aligns the MLP computations with the overall hyperspherical geometry of the model, potentially enhancing

the consistency and interpretability of the network's internal representations.

Transformer	Normalized Transformer
$h_A \leftarrow \text{ATTN}(\text{RMSNorm}(h))$	$h_A \leftarrow \text{Norm}(\text{ATTN}(h))$
$h \leftarrow h + h_A$	$h \leftarrow \text{Norm}(h + \alpha_A(h_A - h))$
$h_M \leftarrow \text{MLP}(\text{RMSNorm}(h))$	$h_M \leftarrow \text{Norm}(\text{MLP}(h))$
$h \leftarrow h + h_M$	$h \leftarrow \text{Norm}(h + \alpha_M(h_M - h))$
Final: $h \leftarrow \text{RMSNorm}(h)$	

Table 1. Transformer vs. Normalized Transformer.

3. Methodology

We have implemented the proposed approach of normalized transformer in a Vision Transformer architecture and we have conducted experiments to rigorously evaluate the performance of our **normalized transformer-based Vision Transformer** (nViT) in comparison to standard **ViT** architectures across a diverse range of computer vision tasks. Our experiments are conducted on five distinct datasets: **MNIST**, **CIFAR-10**, **CIFAR-100**, and **Tiny ImageNet**. This selection allows us to assess the model's performance on tasks ranging from simple digit recognition to complex image classification.

The experiments were conducted on a diverse range of hardware setups depending on where the resources were available. We trained out models on GPUs ranging from NVIDIA RTX 4090, T4, L4, and A100.

3.1. Datasets

In this study, we utilize a diverse set of datasets to evaluate the performance of our normalized transformer-based Vision Transformer (ViT) across various computer vision tasks. The datasets employed in our experiments are:

3.1.1. MNIST

The **MNIST** [5] dataset is a widely-used benchmark in machine learning, particularly for handwritten digit classification tasks. It consists of 28×28 pixels grayscale images of handwritten digits (0 – 9), with 60,000 training images and 10,000 test images.

3.1.2. CIFAR-10 & CIFAR-100

CIFAR-10 and **CIFAR-100** [4] are popular datasets for image classification tasks. Both contain 32×32 pixel color images, with 50,000 training images and 10,000 test images. **CIFAR-10** comprises 10 classes, while **CIFAR-100** includes 100 classes, offering varying levels of complexity for our model evaluation.

3.1.3. Tiny ImageNet

Tiny ImageNet [8] is a challenging dataset derived from the ImageNet database, consisting of 200 classes. Each class

contains 500 training images, 50 validation images, and 50 test images, all resized to 64×64 pixels. This dataset provides a more complex task for our model, testing its ability to classify a larger number of categories with limited spatial information.

3.2. Ablation studies

The central idea for our ablation studies was to evaluate the impact of normalization on transformers. For this, we train all the models mentioned in this study from scratch, i.e. without any pre-training. We start by training both the models on MNIST. Following this, we train 3 iterations of standard ViT: *no normalization*, *batch normalization*, *layer normalization* on both CIFAR-10 and CIFAR-100. In order to train a good standard ViT, we employ a number of data augmentation methods to improve its performance.

Experiments with nViT revealed that adding advanced/more complex augmentations to the datasets leads to a decrease in the model’s performance. Therefore, we train it with all possible combinations of data augmentations in order to rigorously understand the behavior of nViT, and get the best set of augmentations that work best with it. The details are mentioned in table 2.

Due to compute restrictions, we were unable to perform on ImageNet dataset. So, we have experimented on Tiny ImageNet dataset to study performance and convergence on medium sized datasets.

3.2.1. Data Augmentations

1. **Random Crop:** Random Crop is a geometric augmentation technique that randomly extracts a specified region from an input image. In our implementation, we first pads the image with 4 pixels on each side and then crop a region of size equal to the image size (e.g., 32×32 pixels) from the padded image. This randomness ensures that the model sees different portions of the image during training. By cropping different regions, the model learns to recognize objects irrespective of their positions within the image.
2. **Random Horizontal Flip:** This method is an effective augmentation technique that randomly flips an input image horizontally with a default probability of 50%. This transformation mirrors the image along its vertical axis, producing a left-right reversed version of the original image which essentially doubles the number of training samples by introducing mirrored versions of existing images and allows the model to perform better on unseen data by exposing it to varied image orientations during training.
3. **Random Rotation:** Random Rotation is a geometric augmentation technique that rotates an input image by a random angle within a specified range which rotation introduces variability in the orientation of objects within the

image. This enables the model to recognize objects regardless of slight rotations, making it more adaptable to different viewing angles.

4. **Color Jitter:** Color Jitter is a photometric augmentation method that randomly alters the brightness, contrast, saturation, and hue of an input image. This forces the model to focus on shape and texture rather than exact color values, enhancing its ability to recognize objects under different lighting conditions.
5. **Normalize:** Normalize is a photometric transformation that standardizes the pixel values of an image based on specified mean and standard deviation values for each color channel. this transformation adjusts the image data to have zero mean and unit variance per channel. This method ensures that the input data has consistent scaling, which accelerates the convergence of the training process.
6. **Mixup:** MixUp is an advanced data augmentation technique that creates new training samples by linearly combining pairs of images and their corresponding labels. In our experiments, this method blends two images and their labels using a mixing coefficient λ sampled from a Beta distribution. Specifically, $mixed_x = \lambda * x + (1 - \lambda) * x[index, :]$ combines the input images, and $y_a, y_b = y, y[index]$ pairs their labels accordingly. This acts as a form of regularization by preventing the model from becoming overly confident about specific training examples, thereby reducing overfitting, thus helping with generalizations.
7. **ConvStem:** ConvStem is a convolutional stem module integrated at the beginning of the model architecture to process input images before they are fed into transformer blocks. In our experiments, this applies a sequence of two convolutional layers with batch normalization and ReLU activations. This helps with feature extraction as convolutional layers are adept at capturing local patterns such as edges and textures, providing rich feature representations that complement the global attention mechanisms of transformers.

3.2.2. Training method

We use **AdamW** [6] optimizer, which decouples weight decay from the gradient updates, providing more effective regularization and improved convergence compared to traditional Adam optimizers. To dynamically adjust the learning rate during training, we use **Cosine Annealing Learning Rate Scheduler**, which modulates the learning rate following a cosine decay pattern, starting from an initial high value to a lower bound over the course of training epochs which allows the model to converge to an optimal solution. Additionally, we use **Early Stopping**, to halt training when no significant improvement is observed over a predefined number of epochs (patience). This prevents overfitting by ensuring that the model retains the best-performing weights,

thereby maintaining high generalization on unseen data. To further stabilize training and prevent issues like exploding gradients, **Gradient Clipping** is applied using, which restricts the maximum norm of the gradients, ensuring consistent and stable parameter updates.

For the ablations performed on nViT to get an understanding for its behaviour with augmentation methods, we distinguish the ablation studies conducted with *baseline augmentations* and those utilizing *advanced augmentations*. The baseline augmentation ablations employed fundamental data augmentation techniques, including `RandomCrop` and `RandomHorizontalFlip`. The advanced augmentation ablations integrated more sophisticated transformations such as `ColorJitter` and `RandomRotation` along with those in *baseline*. These augmentations are combined with *mixUp* and *ConvStem* as well since both of these sophisticated approaches to augmentation allowed us to get a good accuracy for our ViT. The list of all ablations conducted for nViT is in 2.

For training on MNIST and CIFAR-10, we used a *depth*, *head* = 6, *patch size* = 4, *model dim* = 384, and a batch size of 128 for both the models; for CIFAR-100, we used a *depth*, *head* = 8, *patch size* = 4, *model dim* = 512 for both the models. The standard ViTs were trained for 200 epochs while the nViTs, training for 100 epochs sufficed since they converged faster and also led to triggering early stopping before 100 epochs for some of the runs. For training on TinyImageNet, we used a *depth*, *head* = 9, *patch size* = 8, *model dim* = 384, and a batch size of 128 for both the models. In detail descriptions of our model architectures can be found in our code.

4. Results

4.1. Performance on MNIST & CIFAR-10

Our experiments comparing nViT (normalized Vision Transformer) with the vanilla ViT on CIFAR-10 and MNIST datasets reveal intriguing insights into the performance and efficiency of these architectures. On the CIFAR-10 dataset, nViT demonstrated a consistent improvement in validation accuracy compared to the vanilla ViT. The nViT model achieved a peak validation accuracy of 87.3% after 100 epochs, while the vanilla ViT reached 85.1% under similar training conditions as shown in fig 2. This represents a 2.2 percentage point improvement, suggesting that the normalization techniques employed in nViT contribute to better generalization on complex, diverse datasets like CIFAR-10.

The performance gap between nViT and vanilla ViT was less pronounced on the MNIST dataset, with both models achieving high accuracy. nViT reached a maximum validation accuracy of 99.2%, slightly outperforming the vanilla ViT's 98.9% as shown in fig 1. The smaller difference in performance on MNIST can be attributed to the relative

simplicity of the dataset, which consists of grayscale images of handwritten digits. However, it is noteworthy that nViT still managed to eke out a marginal improvement, suggesting that its normalization techniques provide benefits even on less complex tasks.

These results indicate that the normalization techniques incorporated in nViT offer tangible benefits in terms of model performance and generalization. The more substantial improvement observed on CIFAR-10 implies that nViT's advantages become more pronounced as the complexity of the dataset increases. This suggests that nViT may be particularly well-suited for real-world computer vision tasks that involve diverse and challenging image data. Future work should explore the scalability of nViT to larger datasets and more complex vision tasks to further validate its potential as a robust alternative to vanilla ViT architectures.



Figure 1. Test accuracy for ViT and nViT on MNIST.

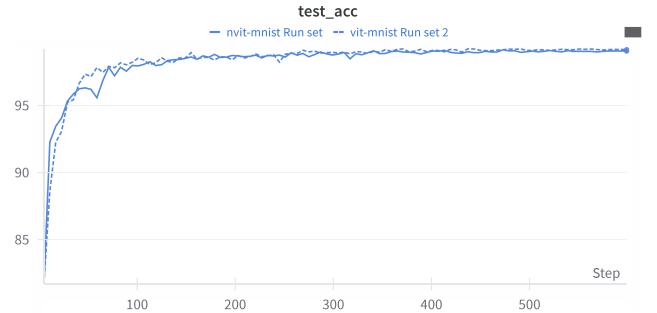


Figure 2. Test accuracy for ViT and nViT on CIFAR-10.

4.2. Ablation on Normalizations in Vision Transformers

We conducted experiments comparing different normalization techniques for Vision Transformers (ViT) on the CIFAR-100 dataset. Fig 3 shows the training loss curves for various ViT configurations, including our proposed

Experiment Name	Use ConvStem	Use MixUp	Augmentation Level
Baseline Augmentations Only(No ConvStem, No Mixup)	No	No	Baseline
MixUp Only (Baseline Augmentations, No ConvStem)	No	Yes	Baseline
ConvStem Only (Baseline Augmentations, No MixUp)	Yes	No	Baseline
ConvStem + MixUp (Baseline Augmentations)	Yes	Yes	Baseline
ConvStem + Advanced Augmentations (No MixUp)	Yes	No	Advanced
Advanced Augmentations only (No ConvStem, No MixUp)	No	No	Advanced
Advanced Augmentations+Mixup(No ConvStem)	No	Yes	Advanced
ConvStem+MixUp(Advanced Augmentations)	Yes	Yes	Advanced

Table 2. List of ablations conducted for nViT

nViT (normalized Vision Transformer). The nViT model, which incorporates the principles of normalized transformers (nGPT), demonstrates significantly faster convergence compared to other variants. Specifically, nViT achieves a lower training loss in fewer iterations, reaching a loss of approximately 0.5 after only 50,000 epochs, while other variants require over 150,000 epochs to approach similar loss values.

The test accuracy results, presented in Fig 4, further corroborate the superiority of nViT. After 500 epochs, nViT achieves the highest test accuracy of 62.47%, outperforming the baseline ViT with standard normalization (57.57%) and variants without normalization (40.36%) or with only batch normalization (26.73%). This substantial improvement in accuracy, coupled with faster convergence, demonstrates that the normalization techniques employed in nViT effectively enhance the model’s learning capacity and generalization ability.

The performance gains of nViT can be attributed to several factors inherent to normalized transformers. By normalizing all vectors and matrices to unit norm, nViT ensures that all computations occur on a hypersphere, leading to more stable and efficient training. The introduction of learnable scaling factors and eigen learning rates allows the model to adaptively control the impact of attention and MLP blocks on the hidden states. This adaptive mechanism, combined with the spherical representation, enables nViT to capture more meaningful features and relationships in the image data, resulting in improved classification performance on CIFAR-100.

4.3. Ablation Study on nViT

We conducted extensive experiments to evaluate the performance of nViT, our normalized Vision Transformer, on the CIFAR-100 dataset. Various ablations were performed to analyze the impact of different components and training strategies. Fig 5 shows the test accuracy curves for different configurations, while Fig 6 presents the corresponding training loss curves. The baseline nViT with ConvStem, MixUp, and advanced augmentations (random crop, horizontal flip, rotation, and color jitter) achieved the highest



Figure 3. Train loss for training ViT with various normalizations along with that of basic nViT.

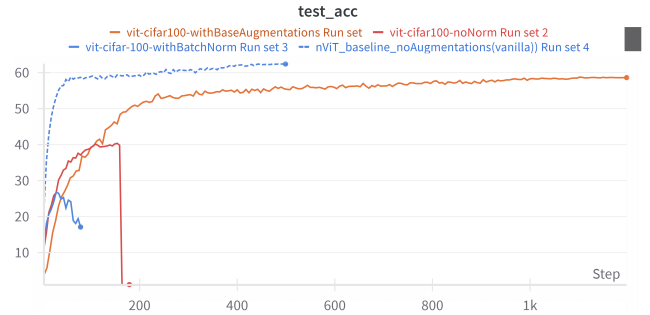


Figure 4. Test accuracy for training ViT with various normalizations along with that of basic nViT

test accuracy of 65.91% after 600 training epochs. This configuration consistently outperformed other variants throughout the training process, demonstrating the effectiveness of combining these techniques. The use of ConvStem appears to be particularly beneficial, as evidenced by the superior performance of models incorporating it compared to those without. Interestingly, the impact of MixUp varies depending on the presence of other components. When used in conjunction with ConvStem and advanced augmentations, MixUp slightly improves performance (65.91% vs. 65.84% at epoch 600). However, in the absence of advanced aug-

mentations, MixUp shows a more significant positive effect (63.56% vs. 61.94% at epoch 600). This suggests that MixUp’s regularization effect is more pronounced when other forms of data augmentation are limited.

The vanilla ViT with MixUp, ConvStem, and advanced augmentations demonstrates competitive performance, albeit with slower convergence compared to nViT variants. It achieves a maximum test accuracy of 65.94% at epoch 988, which is lower than the best nViT configuration. The vanilla ViT’s learning curve shows a steady improvement over time, eventually reaching a comparable accuracy to some nViT configurations. However, it consistently lags behind the best nViT variants in terms of both speed and final accuracy. This underscores the effectiveness of the normalization techniques employed in nViT, which enable faster learning and better overall performance on the CIFAR-100 dataset.

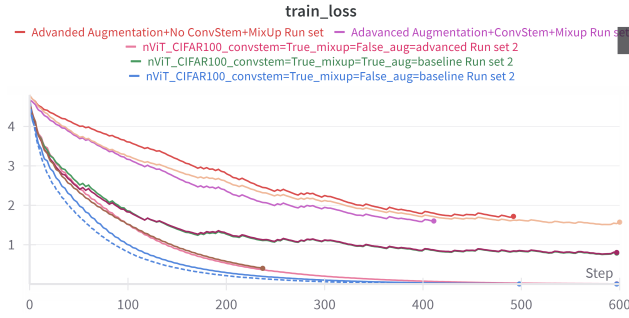


Figure 5. Train loss for ablations on nViT.

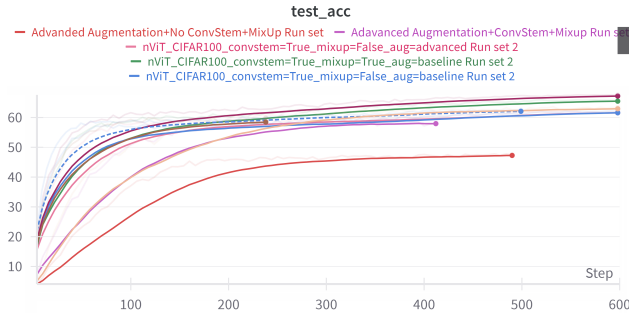


Figure 6. Test accuracy for ablations on nViT.

4.4. Ablations on Tiny Imagenet

We conducted experiments comparing our proposed nViT model, which incorporates normalized transformers (nGPT), against the vanilla ViT on the Tiny ImageNet dataset. Fig 7 illustrates the validation accuracy curves for both models over 80 epochs of training. The nViT model

demonstrates consistently superior performance, achieving a final validation accuracy of 41.82% compared to 39.48% for the vanilla ViT. This represents a significant improvement of 2.34 percentage points. Notably, nViT exhibits faster convergence, reaching 30% accuracy in just 7 epochs, while ViT requires 15 epochs to achieve the same milestone.

The performance gap between nViT and ViT is particularly pronounced in the early stages of training. By epoch 20, nViT attains a validation accuracy of 38.22%, surpassing ViT’s accuracy of 31.98% by a substantial margin of 6.24 percentage points. This early performance boost suggests that the normalized transformer architecture enables more efficient learning of relevant features from the limited Tiny ImageNet dataset. Furthermore, nViT maintains a more stable learning curve throughout training, with fewer fluctuations in validation accuracy compared to the vanilla ViT.

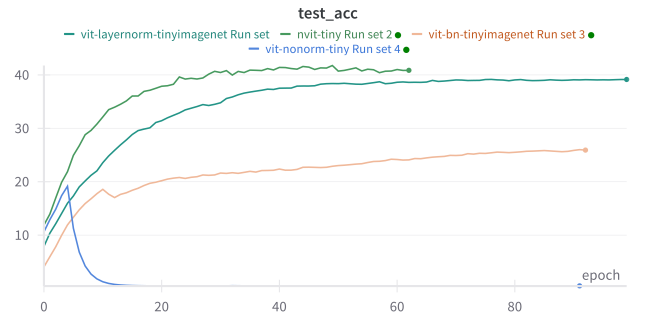


Figure 7. Test accuracy for ablations on Tiny Imagenet.

5. Conclusion & Future Work

Our comprehensive study comparing nViT (normalized Vision Transformer) with vanilla ViT on multiple datasets has yielded insights into the effectiveness of normalization techniques in vision transformers. The results demonstrate consistent performance improvements of nViT across various experimental conditions, suggesting its potential as a robust architecture for computer vision tasks.

On the CIFAR-10 dataset, nViT achieved a modest improvement in validation accuracy compared to vanilla ViT, reaching 87.3% after 100 epochs, while vanilla ViT achieved 85.1%. This 2.2 percentage point difference, while small, indicates a consistent advantage for nViT on this more complex dataset. The performance gap was even less pronounced on the simpler MNIST dataset, with nViT achieving 99.2% accuracy compared to vanilla ViT’s 98.9%. These results suggest that the normalization techniques employed in nViT contribute to slightly better generalization, with the advantage becoming more apparent as dataset complexity increases.

Model	CIFAR-10	MNIST	CIFAR-100	CIFAR-100 (Aug)	Tiny ImageNet
nViT	87.3%	99.2%	62.41%	67.59%	42.18%
ViT (LayerNorm)	85.1%	98.9%	58.77%	66.43%	39.48%

Table 3. Best validation accuracy comparison between nViT and vanilla ViT on different datasets. CIFAR-100 (Aug) represents the best ViT and nViT models trained. For ViT, the best performance was given by the model trained with advanced augmentations, along with mixUp and ConvStem; the best accuracy for nViT was given by using baseline augmentations with mixUp without ConvStem.

The ablation studies on CIFAR-100 provided more substantial evidence of nViT’s advantages. Our proposed model demonstrated significantly faster convergence and higher test accuracy (62.47%) compared to other ViT variants. The combination of unit norm normalization, learnable scaling factors, and eigen learning rates in nViT appears to enhance the model’s learning capacity and stability. Moreover, the integration of ConvStem, MixUp, and advanced augmentation techniques pushed the performance of nViT even further, achieving a peak accuracy of 67.18% on CIFAR-100.

While we extensively experimented on CIFAR 100 and Tiny ImageNet, it is crucial to evaluate its effectiveness on larger, more complex datasets such as ImageNet. This will help assess the scalability of the normalization techniques and their impact on more challenging visual recognition tasks. Given sufficient compute, we plan to do experiments on ImageNet and also pre-train the ImageNet and check it’s finetuning and transfer learning capabilities.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 1
- [2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1
- [3] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015. 1
- [4] Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009. 2
- [5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 2
- [6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2019. 3
- [7] Ilya Loshchilov, Cheng-Ping Hsieh, Simeng Sun, and Boris Ginsburg. ngpt: Normalized transformer with representation learning on the hypersphere. *arXiv preprint arXiv:2410.01131*, 2024. 2
- [8] Stanford University. Tiny imagenet visual recognition challenge. <http://cs231n.stanford.edu/>, 2015. 2

- [9] Yuxin Wu and Kaiming He. Group normalization. *arXiv preprint arXiv:1803.08494*, 2018. 1